

CSE 102 — Fall 2020 – Lab Assignment 2

There is *one* problem in this assignment.

Before you begin the assignment, please read the following carefully.

- Write your answer in the space provided.
- Every part of each question begins on a new page. Do not change this.
- This does not mean that you should write a full page for every question. Your answers should be short and precise. Lengthy and wordy answers will lose points.
- Do not change the format of this document. Simply type your answers as directed.
- The lab is due for submission on Gradescope at the following times
 - All Sections: 03 November (10 a.m.) on Gradescope
- You can discuss these problems amongst each other but the submitted work must be your own.

I have read and agree to the collaboration policy. – Baladithya Balamuurgan, bbalamur@ucsc.edu
Collaborators:

Question I: Assume you are planning a canoe trip down a river. The river has n trading posts numbered 1 to n going downstream. You will start your trip at trading post number 1 and end at trading post number n . Let $R(i, j)$ be the cost of renting a canoe at trading post i and returning it at trading post j , where $j > i$. Assume that you always want to go down river, so the costs if $j \leq i$ are irrelevant. Find the cheapest sequence of rentals that allow you to complete your trip. Aim for an algorithm running in $O(n^2)$ time.

For example, if $n = 4$ and the costs are:

$R(i,j)$	j		
i	2	3	4
1	15	25	35
2	—	12	16
3	—	—	5

then the cheapest sequence of canoe rentals to travel the river would be to rent from 1 to 3, and then from 3 to 4 for a cost of $25 + 5 = 30$.

On the other hand, if the costs were:

$R(i,j)$	j		
i	2	3	4
1	20	15	30
2	—	5	10
3	—	—	20

then taking one canoe all the way from 1 to 4 and renting 1 to 2 and then 2 to 4 are the cheapest solutions (both cost 30). (Note that renting from 1 to 3 is cheaper than going 1 to 2, but the 1 to 3 rental is not in any of the cheapest 1 to 4 solutions.)

Let $C(k)$ be the cost of the cheapest sequence of canoe rentals starting from trading post 1 and returning the last canoe rented at trading post k .

1. (10 pts) Assume an optimal sequence of rentals changes canoes at some trading post j . What sub-problems are also solved optimally by (parts of) this rental sequence? Prove your answer (probably using a proof by contradiction)

Solution.

Assume sequence S is a the optimal sequence of rental changes canoes at some trading post j . We can say that any sub-sequence (contiguous) is also optimal.

From this we can declare optimal subsequences S_1 and S_2 which are 1 to j and j to n respectively.

Proof by Contradiction:

Assume S_1 is **NOT** optimal.

Then there $\exists S'_1$ that is an optimal path from 1 to j .

In turn $S' = \text{concat}(S'_1, S_2)$ and $\text{cost}(S') < \text{cost}(S)$. Then S is not optimal.

However since we assume S to be optimal, this is contradicting the assumption.

Therefore S_1 is optimal.

Similarly:

Assume S_2 is **NOT** optimal.

Then there $\exists S'_2$ that is an optimal path from j to n .

In turn $S' = \text{concat}(S_1, S'_2)$ and $\text{cost}(S') < \text{cost}(S)$. Then S is not optimal.

However since we assume S to be optimal, this is contradicting the assumption.

Therefore S_2 is optimal.

2. (10 pts) Derive a recurrence for $C(k)$ in terms of $C(j)$ values where $j < k$. Explain how you obtained it.

Solution.

$R = 2D$ array of values

$C = 1D$ array with enough elements for the number of signposts

$$C(k) = \begin{cases} 0 & k = 1 \\ \min_{1 \leq j < k} [C(j) + R(j, k)] & 2 \leq k \leq n \end{cases}$$

Explanation:

Since we are trying to find the most optimal path from point A to B we assume that there is a midpoint k from A to $B - 1$ that is part of the path. If the path from A to B is optimal then the subpath(s) (contiguous) are also optimal. Therefore there is a point j from A to $k - 1$ that makes path A to k optimal. Each subpath needs to be the minimum cost to be optimal. Using this logic the above recurrence is calculated.

3. (20 pts) Give a bottom-up iterative algorithm (pseudo code) for computing the $C(j)$ values. This algorithm may also compute “SignPosts” values for use in the following part. It may be helpful to first construct a recursive algorithm for computing the $C(k)$ values. You do not have to prove correctness of the algorithm; briefly state why your algorithm works.

Solution.

Params:

$R = n \times n$ array of values

$n =$ number of trading posts

Returns:

$C = 1 \times n$ array of optimal costs per trading post destination

$SignPosts = 1 \times n$ array of previous post to get to current post optimally

Algorithm 1 Optimal Path

```

1: function MISERCANOETRIP( $R, n$ )
2:    $C \leftarrow$  new Array( $n$ )
3:    $SignPosts \leftarrow$  new Array( $n$ )
4:    $C[0] \leftarrow 0$ 
5:    $SignPosts[0] \leftarrow 0$ 
6:   for  $k=1; k \leq n; k++$  do
7:      $min \leftarrow$  INT32.MAX
8:      $post \leftarrow$  NULL
9:     for  $j=0; j < k; j++$  do
10:       $cost \leftarrow C[j] + R[j][k]$ 
11:      if  $cost < min$  then
12:         $min \leftarrow cost$ 
13:         $post \leftarrow j$ 
14:      end if
15:    end for
16:     $C[k] \leftarrow min$ 
17:     $SignPosts[k] \leftarrow post$ 
18:  end for
19:  return  $C, SignPosts$ 
20: end function

```

Solution.

Explanation:

The initial for loop iterates a value k from the “second” to the last trading post. The secondary for loop then finds the minimum of the combinations of paths from the “first” to the variable k . Rather than recursively calculating it is better to store the values for each sign post as the values are not going to change as they are the most optimal values.

4. (10 pt) Finally, show how keeping a “little” (i.e. $O(n)$) additional information allows the a cheapest sequence of canoe rentals to be printed out in $O(n)$ time.

Solution.

The array SignPosts stores the previous post leading to the current post that is part of the optimal path to the post. We already discussed how the optimal path from A to B has sub paths that are also optimal. Thereby storing the previous post to the current we are acting upon the aforementioned assumption of optimal subpaths. The path to the previous post is optimal and the the path from the previous to the current is then also optimal because that is how the algorithm works. The array is of $O(n)$ space complexity and from that with a single run through we can get the most optimal path from the start to the target destination.

Algorithm 2 Print Optimal Path

```
1: function PRINTMISERCANOETRIP(C, SignPosts, start, destPost)
2:   if destPost == start then
3:     print( destPost + ": " + C[destPost] )
4:     return
5:   end if
6:   prevPost ← SignPosts[destPost]
7:   PrintMiserCanoeTrip(C, SignPosts, start, prevPost)
8:   print( destPost + ": " + C[destPost] )
9:   return
10: end function
```

Solution. *This function takes the results of the Optimal Path algorithm and prints the optimal path from the start to the target destination. The algorithm runs in $O(n)$ time since there is only 1 recursive call not splitting the array(s).*